

## DMA\_project1\_team15 보고서

### 목차

1. 서론
2. Part 1
  - 2.1. 문제 정의
  - 2.2. 도식화 과정
  - 2.3. 최종 ER diagram
3. Part 2
  - 3.1. 문제 정의
  - 3.2. 설계한 스키마 및 코드 설명
4. 결론
5. 참고 문헌

## 1. 서론

본 보고서는 온라인 Q&A 커뮤니티 서비스 A의 데이터베이스를 설계 및 구현하는 과정을 기술한다. Part 1에서는 10가지의 requirement들을 만족하는 서비스 A의 DB를 ER diagram을 통해 도식화하였으며, Part 2에서는 주어진 데이터에 적합한 DB schema를 MySQL을 통하여 설계하고, Python을 이용하여 데이터를 입력하였다.

## 2. Part 1

### 2.1. 문제 정의

서비스 A는 온라인 Q&A 커뮤니티 서비스이다. 사용자는 질문을 작성할 수 있으며, 다른 사용자는 이에 대해 답변을 작성할 수 있다. 게시물에는 댓글을 달 수 있고 투표를 생성할 수 있다. 또한 게시물에는 태그가 부여될 수 있고 사용자에게는 여러 개의 배지가 주어질 수 있다. 게시물의 수정 이력은 기록되며, 게시물 간 연결 관계가 존재한다.

본 절에서는 이러한 서비스 A가 만족해야 하는 10가지 영역의 요구사항을 분석하여 ER diagram을 그리는 것을 목적으로 한다. ER diagram 도식화 과정은 ① Requirements 분석(제시된 10개의 요구사항에서 초기 Entity, Attribute, Relationship 식별) - ② 설계 결정(수정 및 보완 과정) - ③ Cardinality ratio 및 participation constraints 확정의 3단계로 진행하였다.

본 보고서의 ER diagram의 relationship constraints는 cardinality ratio(1:1, 1:N, M:N)와 participation constraints(partial, total)로 표현하였으며 (min, max) notation은 사용하지 않았다. ER diagram의 Entity와 Attribute, Relationship 이름 및 Cardinality Ratio는 주어진 데이터를 참고하여 작성하였으며, cardinality ratio 등을 확정할 때 Part 2의 일부 데이터를 참고하기도 하였다. Primary Key는 PK로 표현하였고 ER diagram의 표기법은 강의자료의 표기에 따랐으며, 최종 ER diagram은 draw.io를 활용하여 작성하였다.

### 2.2. 도식화 과정

#### ① Requirements 분석

| Requirement      | Entity   | Attributes                                                                                            | Relationship                                                                   |
|------------------|----------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| (R1-1)<br>사용자 정보 | Users    | UserId(PK), Reputation, DisplayName, Age, CreationDate, LastAccessDate, WebsiteUrl, Location, AboutMe |                                                                                |
| (R1-2)<br>게시물 정보 | Posts    | Id(PK), CreationDate, Content, OwnerUserId, LastActivityDate                                          | WRITES (Users(partial) -> Posts(partial)), 1:N<br>IS_QUESTION ?<br>IS_ANSWER ? |
| (R1-3)           | Question | Id(PK), PostId, ViewCount,                                                                            | HAS(Question(partial) ->                                                       |

|                            |                           |                                                              |                                                                                                                                                                 |
|----------------------------|---------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 질문 정보                      |                           | Title, Tags,<br>AcceptedAnswerId                             | Answer(total)), 1:N<br>ACCEPTED(Question(partial)<br>-> Answer(partial)), 1:1                                                                                   |
| (R1-4)<br>답변 정보            | Answer                    | Id(PK), PostId, Accepted,<br>ParentId                        |                                                                                                                                                                 |
| (R1-5)<br>댓글 정보            | Comments<br>(Weak entity) | ID(partial key), Score,<br>CreationDate                      | <b>Identifying relationship</b><br>HAS_COMMENT<br>(Posts(partial) -><br>Comments(total)), 1:N,<br>WRITES_COMMENT<br>(Users(partial) -><br>Comments(total)), 1:N |
| (R1-6)<br>투표 정보            | Votes                     | Id(PK), VoteTypeId,<br>CreationDate,<br>BountyAmount         | HAS_VOTES (Posts(partial) -><br>Votes(total)), 1:N<br>VOTES_BY (Users(partial) -><br>Votes(partial)), 1:N                                                       |
| (R1-7)<br>태그 정보            | Tags                      | Id(PK), TagName,<br>ExcerptPostId, WikiPostId                | HAS_TAG(Question(partial)<br><-> Tags(total)), M:N                                                                                                              |
| (R1-8)<br>배지 정보            | Badges                    | Id(PK), Name, Date                                           | HAS_BADGES(Users(partial)<br>-> Badges(total)), 1:N                                                                                                             |
| (R1-9)<br>수정 및 이력<br>정보    | PostHistory               | Id(PK), PostHistoryTypeId,<br>CreationDate, Text,<br>Comment | HAS_HISTORY (Posts(partial)<br>-> PostHistory(total)), 1:N<br>PERFORMED_BY<br>(Users(partial) -><br>PostHistory(partial)), 1:N                                  |
| (R1-10)<br>게시물 간의<br>연결 정보 | Post_Links                | Id(PK), CreationDate,<br>LinkTypeId                          | LINKS (Posts -> Post_Links),<br>M:N<br>LINKED_TO (Post_Links -><br>Posts), M:N                                                                                  |

## ② 설계 결정

**(R1-2), (R1-3), (R1-4):** (R1-2)에 따라 게시물은 질문 또는 답변 중 하나에만 속해야 하는 상호 배타적(Disjoint) 제약조건을 가진다. EER의 specialization 기호를 사용할 수 없는 제약이 있어, 초기 설계에서는 ER 모델에서 Posts 엔티티에 별도의 구분자 속성을 추가하여 해결하려고 하였다. 그러나 본 프로젝트의 PART II (R2-2) 요구사항인 '주어진 데이터셋의 테이블 및 컬럼과 일치해야 한다'는 물리적 제약을 준수하기 위해, 인위적인 구분자 속성 추가를 배제하고, 1:1 relationship을 활용하여 다음과 같이 게시물과 질문, 게시물과 답변 사이의 관계를 설계하고 다이어그램 상에 텍스트 제약조건을 명시하여 논리적 배타성을 표현했다. 한편, 실제 데이터는 question.csv와 answer.csv에 존재하는 Foreign Key의 존재 여부 자체가 암묵적인 구분자 역할을 수행하도록 설계하였다.

Relationship:

IS\_QUESTION (Posts(partial) -> Question(total)), 1:1

IS\_ANSWER (Posts(partial) -> Answer(total)), 1:1

[Constraint] 어떤 Posts도 Question과 Answer에 동시에 속할 수 없다.

(R1-10) 초기에는 Post\_Links를 Entity로 설계하였으나, 최종 ER diagram 도식화 과정에서  
 (i) Post\_Links는 독립적인 존재 의미를 갖는다고보다 두 게시물 사이의 연결 관계 자체를 표현하는 성격이 강하다는 점,  
 (ii) Post\_Links의 속성은 두 게시물의 조합에 의해 결정되는 relationship으로 표현이 가능하다는 점,  
 (iii) Entity로 설계할 경우 불필요한 중간 과정으로 인해 모델의 복잡도가 증가한다는 점 등을 고려하여 다음과 같은 Posts간의 M:N recursive relationship으로 변경하였다.  
 Relationship: POST\_LINKS (Posts(partial) -> Posts(partial)), M:N

(R1-7) 데이터의 태그 테이블에 ExcerptPostId, WikiPostId가 있는 것을 확인하고, Tags entity와 Posts entity 사이의 Relationship을 정의하여 ER diagram에 반영하였다.

Relationship:

HAS\_EXCERPT(Tags(partial) -> Posts(partial)), 1:1

HAS\_WIKI(Tags(partial) -> Posts(partial)), 1:1

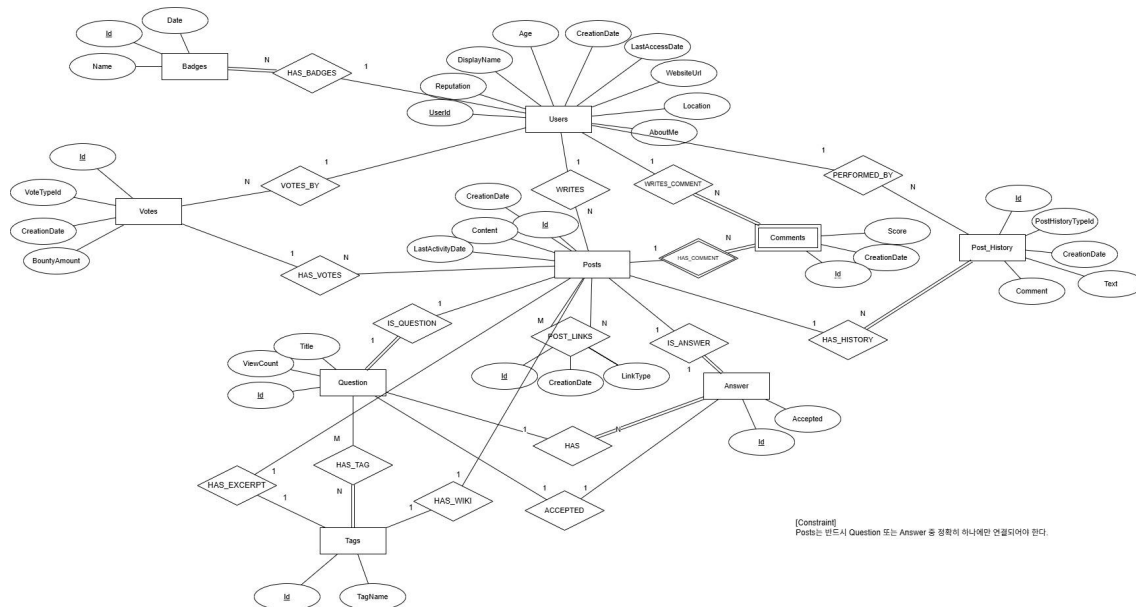
### ③ 최종 Cardinality ratio 및 participation constraints와 그 근거

| Relationship                                                          | Cardinality ratio                    | Participation Constraints                                                          |
|-----------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------------------------------|
| WRITES<br>(Users -> Posts)                                            | 1:N<br>(한 사용자가 여러 개의 게시물에 대응되므로)     | Users(Partial)<br>Posts(Partial)<br>(일부 게시물은 작성자 정보가 없을 수도 있다고 하였으므로 partial)      |
| IS_QUESTION<br>(Posts -> Question)                                    | 1:1                                  | Posts(Partial)<br>Question(total)<br>(모든 질문은 게시물에 대응되지만, 게시물은 질문이 아닐 수도 있으므로)      |
| IS_ANSWER<br>(Posts -> Answer)                                        | 1:1                                  | Posts(Partial)<br>Answer(total)<br>(마찬가지로, 모든 답변은 게시물에 대응되지만 게시물은 답변이 아닐 수도 있다)    |
| HAS<br>(Question -> Answer)                                           | 1:N<br>(하나의 질문은 여러 개의 답변을 가질 수 있으므로) | Question(partial)<br>Answer(total)<br>(답변은 반드시 하나의 질문에 속해야 하며, 질문은 답변이 없을 수도 있으므로) |
| ACCEPTED<br>(Question -> Answer)                                      | 1:1                                  | Question(partial)<br>Answer(partial)<br>(채택 답변이 없을 수도 있고, 모든 답변이 채택되는 것은 아니므로)     |
| HAS_COMMENT<br>(Posts -> Comments)<br><b>Identifying relationship</b> | 1:N                                  | Posts(partial)<br>Comments(total)<br>(댓글은 반드시 특정 게시물에 달리지만, 게시물은 댓글이 없을 수도 있으므로)   |

|                                       |                                                       |                                                                                                      |
|---------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| WRITES_COMMENT<br>(Users → Comments)  | 1:N                                                   | Users(partial)<br>Comments(total)<br>(댓글은 반드시 특정 사용자에게 의해 작성되지만, 사용자는 댓글을 달지 않을 수 있으므로)              |
| HAS_VOTES<br>(Posts → Votes)          | 1:N                                                   | Posts(partial)<br>Votes(total)<br>(투표는 특정 게시물에 대해 이루어지지만, 게시물에는 투표가 없을 수도 있으므로)                      |
| VOTES_BY<br>(Users → Votes)           | 1:N                                                   | Users(partial)<br>Votes(partial)<br>(투표는 특정 사용자가 남긴 것일 수도 있고 아닐 수도 있으므로)                             |
| HAS_TAG<br>(Question <-> Tags)        | M:N<br>(하나의 질문에 여러 개의 태그가, 하나의 태그가 여러 질문에 사용될 수 있으므로) | Question(partial)<br>Tags(total)<br>(태그는 질문에 부여되지만, 질문에 반드시 태그가 있어야 하는 것은 아니므로)                      |
| HAS_EXCERPT (Tags → Posts)            | 1:1                                                   | Tags(partial)<br>Posts(partial)<br>(태그에 반드시 excerpt 게시물이 있는 것은 아니며, 모든 게시물이 어떤 태그의 excerpt인 것도 아니므로) |
| HAS_WIKI (Tags → Posts)               | 1:1                                                   | Tags(partial)<br>Posts(partial)<br>(태그에 반드시 wiki 게시물이 있는 것은 아니며, 모든 게시물이 어떤 태그의 wiki인 것도 아니므로)       |
| HAS_BADGES<br>(Users → Badges)        | 1:N                                                   | Users(partial)<br>Badges(total)<br>(배지는 반드시 특정 사용자에게 속하지만, 사용자는 배지가 없을 수도 있으므로)                      |
| HAS_HISTORY<br>(Posts → PostHistory)  | 1:N                                                   | Posts(partial)<br>PostHistory(total)<br>(모든 이력은 특정 게시물에 속하지만, 게시물은 이력이 없을 수도 있으므로)                   |
| PERFORMED_BY<br>(Users → PostHistory) | 1:N                                                   | Users(partial)<br>PostHistory(partial)<br>(이력 수정을 특정 사용자가 수행했을 수도 있고 아닐 수도 있으므로)                     |
| POST_LINKS<br>(Posts → Posts)         | M:N                                                   | Posts(partial)<br>Posts(partial)<br>(연결이 안 된 게시물이 존재할 수 있으므로 partial)                                |

- ER diagram의 구조상 다른 Entity를 참조하는 Attribute는 diagram에 표시하지 않고, relationship으로 표현된다. 예를 들어, Post의 속성인 작성자 정보(OwnerUserId)는 Users를 참조하므로, diagram에서는 WRITES relationship으로 대체하여 표현하였다.

## 2.3. 최종 ER diagram



### 3. Part 2

#### 3.1. 문제 정의

본 프로젝트의 Part 2는 앞서 설계한 ER diagram을 바탕으로 실제로 구현 가능한 관계형 스키마(MySQL)를 설계하고, 이를 Python으로 구현하는 것을 목적으로 한다. requirement에서 없을 수 있다고 명시된 속성에 대해서 NULL 허용 여부를 결정하였다. 또한 M:N relationship의 경우 별도의 테이블(postLinks)를 활용하여 표현하였으며, relationship으로 표현된 entity 간의 연결을 Foreign key constraint로 변환하였다.

Part 2는 Requirement에 따라 다음과 같은 네 단계로 진행하였다. ① Schema 생성(R2-1) - ② 데이터 저장을 위한 table 10개 생성(R2-2) - ③ 주어진 csv 파일의 데이터 입력(R2-3) - ④ 테이블 간 Foreign Key 제약조건 추가(R2-4).

본 보고서에는 작성한 모든 코드를 포함하지 않고 일부 주요 코드에 대한 설명만 작성하였다. DB의 구현을 위해서 Python의 mysql-connector-python 라이브러리와 csv 라이브러리만 사용하였으며, Python은 3.14.3 버전을 사용하였다. 사용한 데이터는 “2026-1 데관분 프로젝트-1.zip” 내부 10개 csv 파일이다.

#### 3.2. 설계한 스키마 및 코드 설명

##### ① Schema 생성(R2-1)

먼저 다음과 같은 코드를 사용하여 DMA\_team15 schema를 생성하였다. 스키마가 존재할 경우 생성 과정을 다시 수행하지 않아야 하므로 CREATE SCHEMA IF NOT EXISTS DMA\_team15; 를 사용하였다.

```
def requirement1(host, user, password):
    cnx = mysql.connector.connect(host=host, user=user, password=password)
    cursor = cnx.cursor()
    cursor.execute('SET GLOBAL innodb_buffer_pool_size=2*1024*1024*1024;')
    print('Creating schema...')

    cursor.execute('CREATE SCHEMA IF NOT EXISTS DMA_team15;')
    cnx.commit()

    cursor.close()
    cnx.close()
```

##### ② 테이블 설계(R2-2)

각 테이블에 대하여 명시된 조건에 맞추어 데이터 타입을 매핑하고 테이블을 생성하였다. 생성하는 table과 column의 이름은 csv 파일의 헤더와 일치하도록 하였으며, 고유번호에는 PK를 부여하였다. 이 단계에서 설계한 스키마는 다음과 같다.

##### (1) Users

| Column         | Type                  |
|----------------|-----------------------|
| UserId(PK)     | INT(11) NOT NULL      |
| Reputation     | INT(11) NOT NULL      |
| DisplayName    | VARCHAR(255) NOT NULL |
| Age            | INT(11)               |
| CreationDate   | DATETIME NOT NULL     |
| LastAccessDate | DATETIME NOT NULL     |
| WebsiteUrl     | VARCHAR(255)          |
| Location       | VARCHAR(255)          |
| AboutMe        | LONGTEXT              |

## (2) Posts

| Column          | Type              |
|-----------------|-------------------|
| Id(PK)          | INT(11) NOT NULL  |
| CreationDate    | DATETIME NOT NULL |
| Content         | LONGTEXT NOT NULL |
| OwnerUserId     | INT(11)           |
| LasActivityDate | DATETIME NOT NULL |

## (3) Question

| Column           | Type                  |
|------------------|-----------------------|
| Id(PK)           | INT(11) NOT NULL      |
| PostId           | INT(11) NOT NULL      |
| AcceptedAnswerId | INT(11)               |
| ViewCount        | INT(11) NOT NULL      |
| Title            | VARCHAR(255) NOT NULL |
| Tags             | VARCHAR(255) NOT NULL |

## (4) Answer

| Column   | Type             |
|----------|------------------|
| Id(PK)   | INT(11) NOT NULL |
| PostId   | INT(11) NOT NULL |
| Accepted | INT(11) NOT NULL |
| ParentId | INT(11) NOT NULL |

## (5) Comments

| Column          | Type              |
|-----------------|-------------------|
| Id(Partial Key) | INT(11) NOT NULL  |
| PostId          | INT(11) NOT NULL  |
| Score           | INT(11) NOT NULL  |
| CreationDate    | DATETIME NOT NULL |
| UserInId        | INT(11) NOT NULL  |

## (6) Votes



| Column       | Type              |
|--------------|-------------------|
| Id(PK)       | INT(11) NOT NULL  |
| PostId       | INT(11) NOT NULL  |
| VoteTypeId   | INT(11) NOT NULL  |
| CreationDate | DATETIME NOT NULL |
| UserInId     | INT(11)           |
| BountyAmount | INT(11)           |

## (7) Tags

| Column        | Type                  |
|---------------|-----------------------|
| Id(PK)        | INT(11) NOT NULL      |
| TagName       | VARCHAR(255) NOT NULL |
| ExcerptPostId | INT(11)               |
| WikiPostId    | INT(11)               |

## (8) Badges

| Column   | Type                  |
|----------|-----------------------|
| Id(PK)   | INT(11) NOT NULL      |
| UserInId | INT(11) NOT NULL      |
| Name     | VARCHAR(255) NOT NULL |
| Date     | DATETIME NOT NULL     |

## (9) PostHistory

| Column            | Type              |
|-------------------|-------------------|
| Id(PK)            | INT(11) NOT NULL  |
| PostHistoryTypeId | INT(11) NOT NULL  |
| PostId            | INT(11) NOT NULL  |
| CreationDate      | DATETIME NOT NULL |
| UserInId          | INT(11)           |
| Text              | LONGTEXT          |
| Comment           | LONGTEXT          |

## (10) postLinks

| Column        | Type              |
|---------------|-------------------|
| Id(PK)        | INT(11) NOT NULL  |
| CreationDate  | DATETIME NOT NULL |
| PostId        | INT(11) NOT NULL  |
| RelatedPostId | INT(11) NOT NULL  |
| LinkTypeId    | INT(11) NOT NULL  |

## - 주요 코드 설명

- CREATE TABLE IF NOT EXISTS로 10개 테이블을 순차적으로 생성하고, 데이터 타입 매핑을 진행한다.
- 고유번호(Id, UserId 등)에는 PRIMARY KEY를 부여한다.
- UNIQUE 제약을 사용하여 1:1 관계 부여한다.  
: question 테이블의 PostId 및 AcceptedAnswerId, answer 테이블의 PostId, tags 테이블의 TagId

블의 ExcerptPostId 및 WikiPostId

- NULL값이 가능하지 않은 column에는 NOT NULL 제약을 추가한다.

```

...
# posts
cursor.execute("""
    CREATE TABLE IF NOT EXISTS posts (
        Id INT(11) NOT NULL,
        CreationDate DATETIME NOT NULL,
        Content LONGTEXT NOT NULL,
        OwnerUserId INT(11),
        LasActivityDate DATETIME NOT NULL,
        PRIMARY KEY (Id)
    )
""")

# question
cursor.execute("""
    CREATE TABLE IF NOT EXISTS question (
        Id INT(11) NOT NULL,
        PostId INT(11) NOT NULL,
        AcceptedAnswerId INT(11),
        ViewCount INT(11) NOT NULL,
        Title VARCHAR(255) NOT NULL,
        Tags VARCHAR(255) NOT NULL,
        PRIMARY KEY (Id),
        UNIQUE (PostId),
        UNIQUE (AcceptedAnswerId)
    )
""")
...

```

### ③ 주어진 csv 파일의 데이터 입력(R2-3)

Python 내부의 csv 모듈로 csv 파일을 가져와 각 테이블에 삽입하는 단계로, 데이터 전처리는 다음과 같이 수행하였다. 날짜는 YYYY.M.D HH:MM형태를 YYYY-MM-DD HH:MM:SS 형태로 변환하였으며, csv에서 읽은 값을 문자열 또는 정수로 변환하는데 만약 값이 비어있거나 None이라면 MySQL의 NULL로 변환하였다. 이 과정에서 csv 파일의 데이터를 직접 변형하지 않았다.

한편, comments.csv 데이터 검토 결과 동일 PostId 내 동일 Id를 가지는 레코드가 1,727건 존재함을 확인하였다. 이는 (R1-5)의 '게시물 내 댓글 순번' 해석과 부합하지 않는 데이터이므로, Weak entity의 (Id, PostId) 복합키 제약으로 인해 데이터를 임의로 수정하지 않고 중복 행을 삽입하지 않아 삽입된 데이터 건수가 줄어들었다. 이 단계에서 테이블별 삽입 현황은 다음과 같다.

| 테이블(csv 파일의 튜플 수)    | 삽입된 데이터 건수 |
|----------------------|------------|
| users(40,325)        | 40,325     |
| posts(91,977)        | 91,977     |
| question(42,921)     | 42,921     |
| answer(47,756)       | 47,756     |
| comments(171,467)    | 169,740    |
| votes(323,233)       | 323,233    |
| tags(1,032)          | 1,032      |
| badges(79,851)       | 79,851     |
| postHistory(275,281) | 275,279    |
| postLinks(11,102)    | 11,102     |

- 주요 코드 설명

- flush 함수 정의 : 대량 데이터를 DB에 삽입하기 위하여 배치 삽입 전략을 사용하였다. 1,000단위로 삽입을 진행하고, 오류가 발생하였다면 rollback 후 한 줄씩 삽입을 시도한다. 만약 오류가 또 발생하였다면 그 데이터는 버리고 다음 줄로 넘어간다.

```
def flush(cursor, cnx, sql, batch):
    if batch:
        try:
            cursor.executemany(sql, batch)
        except mysql.connector.Error:
            cnx.rollback()

        for row in batch:
            try:
                cursor.execute(sql, row)
            except mysql.connector.Error:
                pass
        cnx.commit()
```

- 다음은 csv 파일을 읽어온 후, 타입을 변환하고 데이터베이스에 삽입하는 과정이다.

```
def read_csv(filepath):
    return csv.reader(open(filepath, 'r', encoding='utf-8', errors='replace'))
```

- Users 테이블에 데이터를 입력한다. csv 파일을 열고 헤더를 건너뛰는다.

```
reader = read_csv(directory + 'users.csv')
next(reader)
```

- 각 column에 대해 전처리를 진행한다. 빈 값이 올 수 있는 column은 NULL로 변환하고, 날짜 형식은 DATETIME 형식으로 변환하여 삽입한다.

```
for row in reader:
    batch.append((
        int(row[0]),          # UserId
```

```

int(row[1]),          # Reputation
row[2],              # DisplayName
to_int(row[3]),      # Age
convert_dot_date(row[4]), # CreationDate
...

```

- Batch 방식으로 데이터를 삽입하는 코드이다. 데이터가 BATCH\_SIZE에 도달할 때마다 flush하고, 남은 데이터는 최종적으로 한 번 더 flush한다.

```

if len(batch) >= BATCH_SIZE:
    flush(cursor, cnx, 'INSERT INTO users VALUES (...)', batch)
...

```

#### ④ 테이블 간 Foreign Key 제약조건 추가(R2-4)

데이터가 모두 입력된 후, ALTER TABLE을 활용하여 Foreign Key 조건을 추가한다. 이는 referential integrity constraint 위반을 방지하기 위함으로, 참조되는 테이블에 먼저 데이터가 존재해야 하므로 R2-3에서 데이터를 먼저 삽입하였다. 각 테이블의 FK는 Part 1에서 설계한 ER diagram의 relationship을 기반으로 설정하였으며, 구체적인 내용은 다음과 같다.

| 테이블         | FK               | 참조 테이블(키)     | 근거 Relationship |
|-------------|------------------|---------------|-----------------|
| posts       | OwnerUserId      | users(UserId) | WRITES          |
| question    | PostId           | posts(Id)     | IS_QUESTION     |
| question    | AcceptedAnswerId | answer(Id)    | ACCEPTED        |
| answer      | PostId           | posts(Id)     | IS_ANSWER       |
| answer      | ParentId         | question(Id)  | HAS             |
| comments    | PostId           | posts(Id)     | HAS_COMMENT     |
| comments    | UserInId         | users(UserId) | WRITES_COMMENT  |
| votes       | PostId           | posts(Id)     | HAS_VOTES       |
| votes       | UserInId         | users(UserId) | VOTES_BY        |
| tags        | ExcerptPostId    | posts(Id)     | HAS_EXCERPT     |
| tags        | WikiPostId       | posts(Id)     | HAS_WIKI        |
| badges      | UserInId         | users(UserId) | HAS_BADGES      |
| postHistory | PostId           | posts(Id)     | HAS_HISTORY     |
| postHistory | UserInId         | users(UserId) | PERFORMED_BY    |
| postLinks   | PostId           | posts(Id)     | POST_LINKS      |
| postLinks   | RelatedPostId    | posts(Id)     | POST_LINKS      |

#### - 주요 코드 설명

- posts 테이블의 OwnerUserId가 users 테이블의 UserId를 참조하도록 FK 제약을 추가하는 코드이다. 나머지 FK 제약도 다음과 같은 방식으로 순차적으로 추가하였다.

```
'ALTER TABLE posts ADD FOREIGN KEY (OwnerUserId) REFERENCES users(UserId)',
```

위의 내용을 종합하여, 구현한 최종 DB 스키마는 다음과 같은 코드로 구현된다.

[https://drive.google.com/file/d/1wlezWvdUDScRV4pbDUVKXVxNDEDAaQKr/view?usp=drive\\_link](https://drive.google.com/file/d/1wlezWvdUDScRV4pbDUVKXVxNDEDAaQKr/view?usp=drive_link)

#### 4. 결론

본 프로젝트에서는 요구사항으로부터 총 9개의 entity와 16개의 relationship을 설계하여 ER diagram을 도출하고 이를 관계형 DB로 구현했다. 그러나 ER diagram을 작성하는 과정에서 일부 제약조건이 구조적으로 완전하게 표현되지 못하였다는 점(Post와 Question, Answer 간의 관계에서의 상호 배타성이 ER 다이어그램으로 완전히 표현되지 못함), 데이터 입력 과정에서 일부 이상치를 제외하였으나 이 과정에서 데이터 손실이 발생할 수 있다는 점에서 개선이 필요하다. 따라서 향후 정교한 스키마 설계와 데이터 정제를 통해 보다 일관적인 데이터베이스를 구축할 필요가 있을 것으로 보인다.

#### 5. 참고 문헌

데이터관리와 분석 수업 자료(PPT 2,3)

데이터관리와 분석 TA 세션 자료